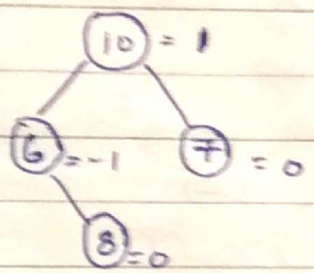
Height Balanced Tree or AVL tree

In this, we compute balance of every node & balance should be $\{-1, 0, 1\}$

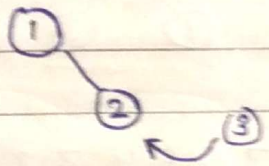
Balance of a node = Height of Left subtree − Height of right subtree



To balance the tree, we perform rotation

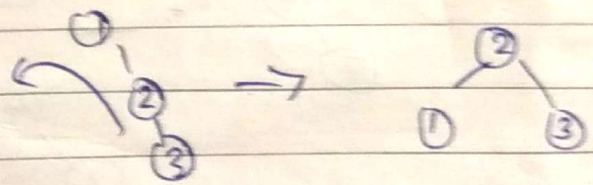
Creating AVL tree

1 2 3 4 5 6 7

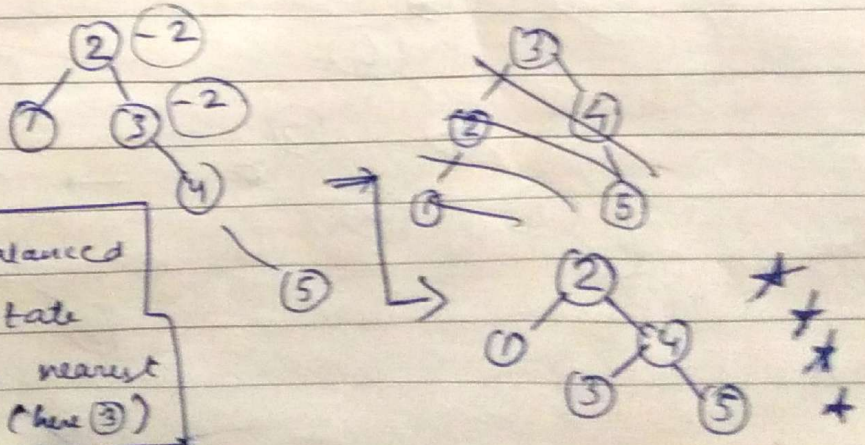


X Not allowed 1 ka balance -2 ho gya

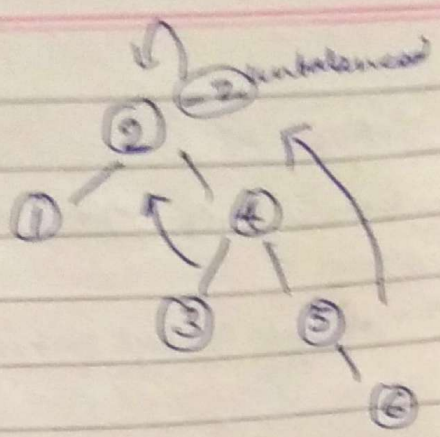
● '1' Right High ho gya h to left rot^n kr do



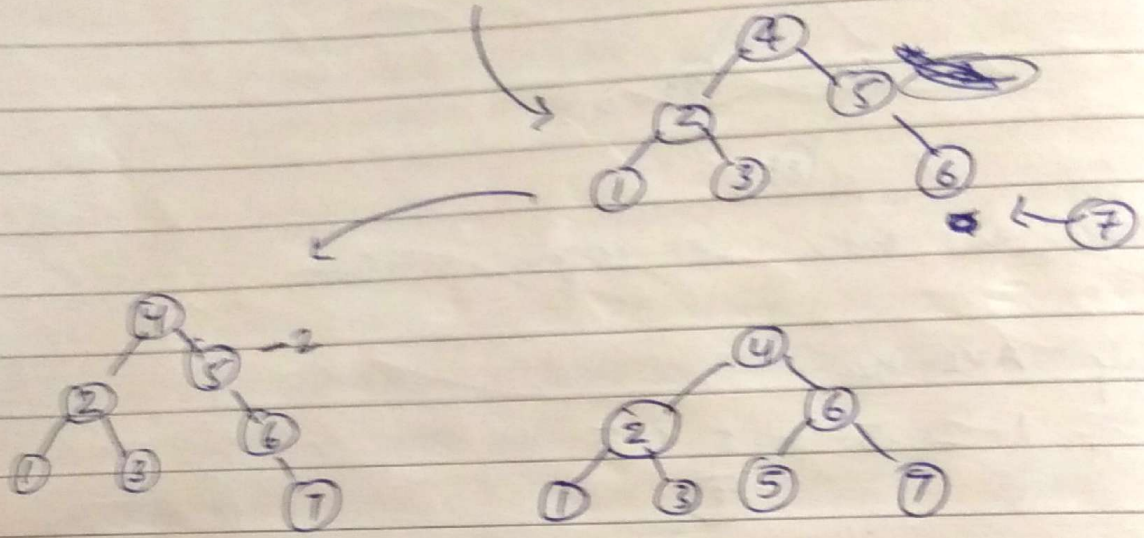
This Rot^n is RR
Coz insertion Right of Right me hua tha



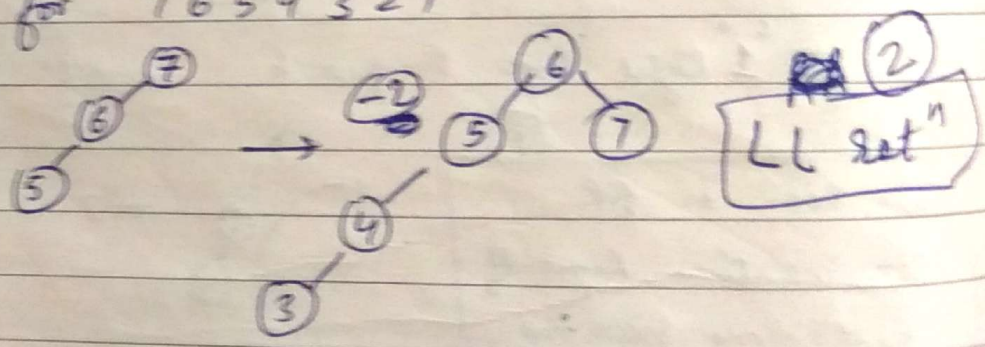
2, 3 dono ka balanced -2 ho gya to, here we rotate the node which is nearest to insertion point (here ③)

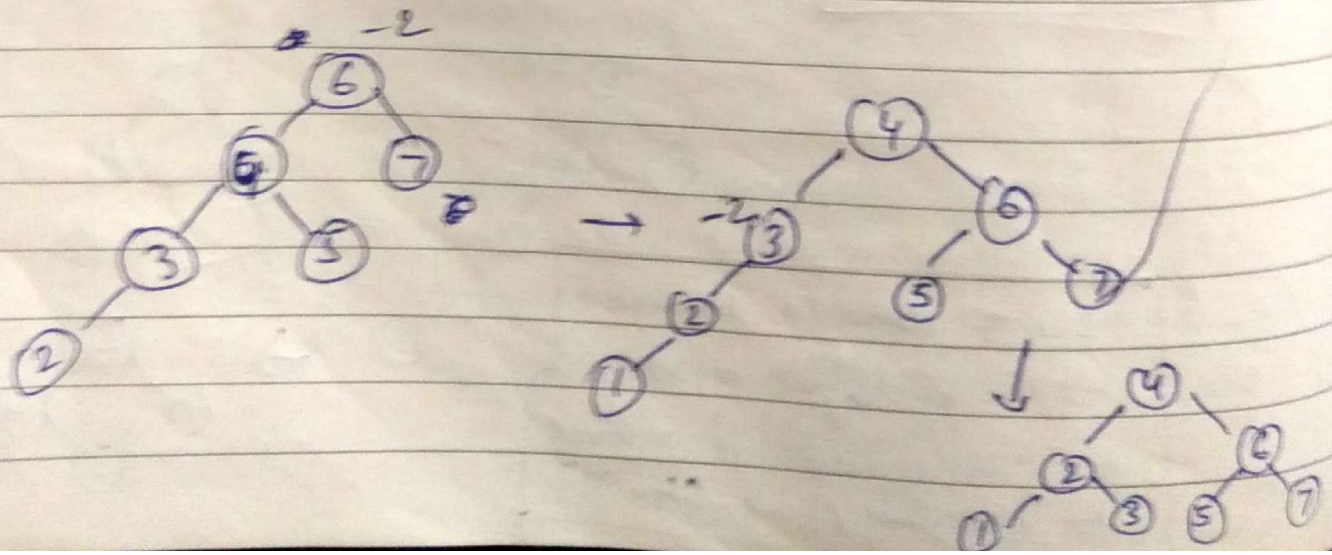


4 ka left child 3 ka right
bn jaega



Draw AVL for 7654321

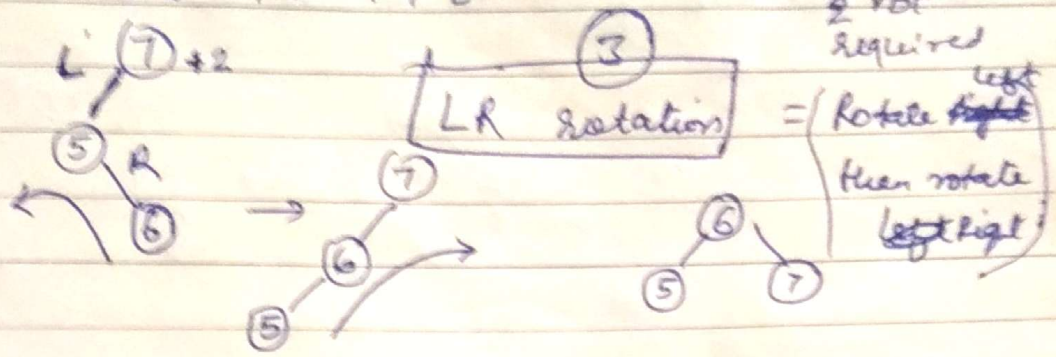


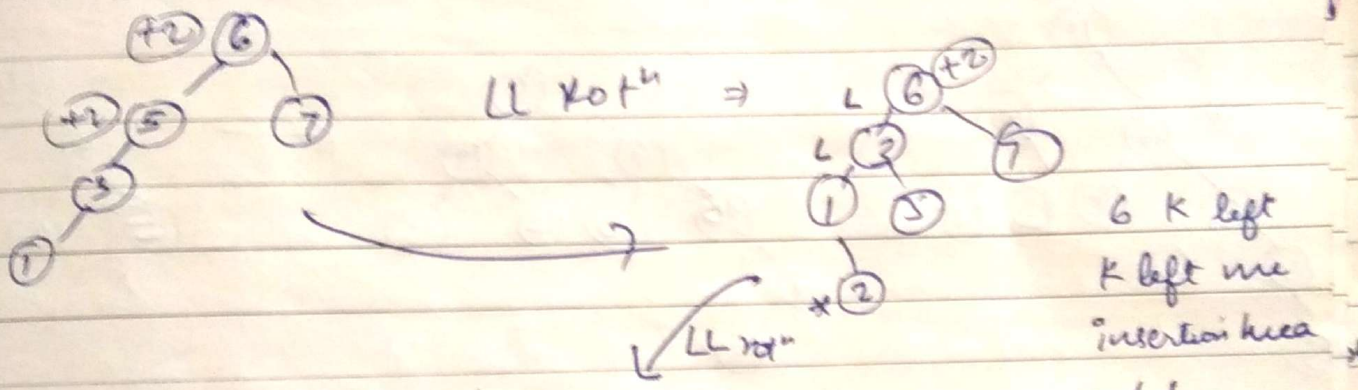


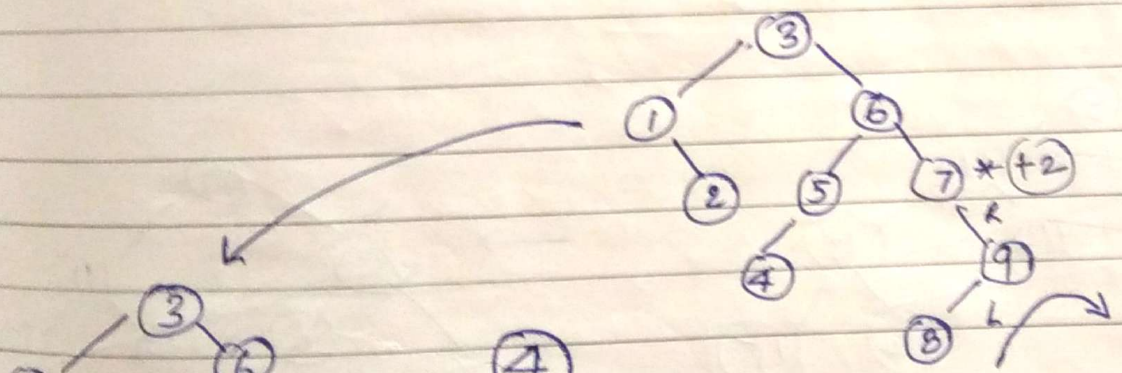
LR & RL require 2 rot^{n}

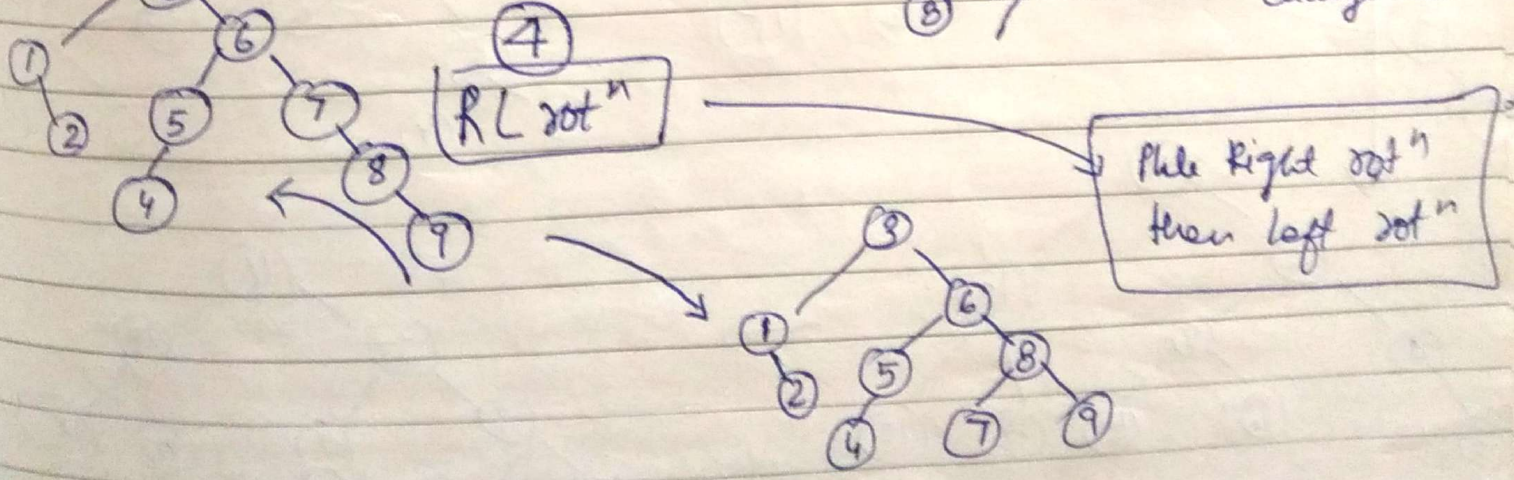
LR → Right Rotate Parent
left rotate child

Create AVL for 7 5 6 3 1 2 4 9 8





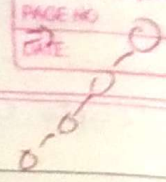




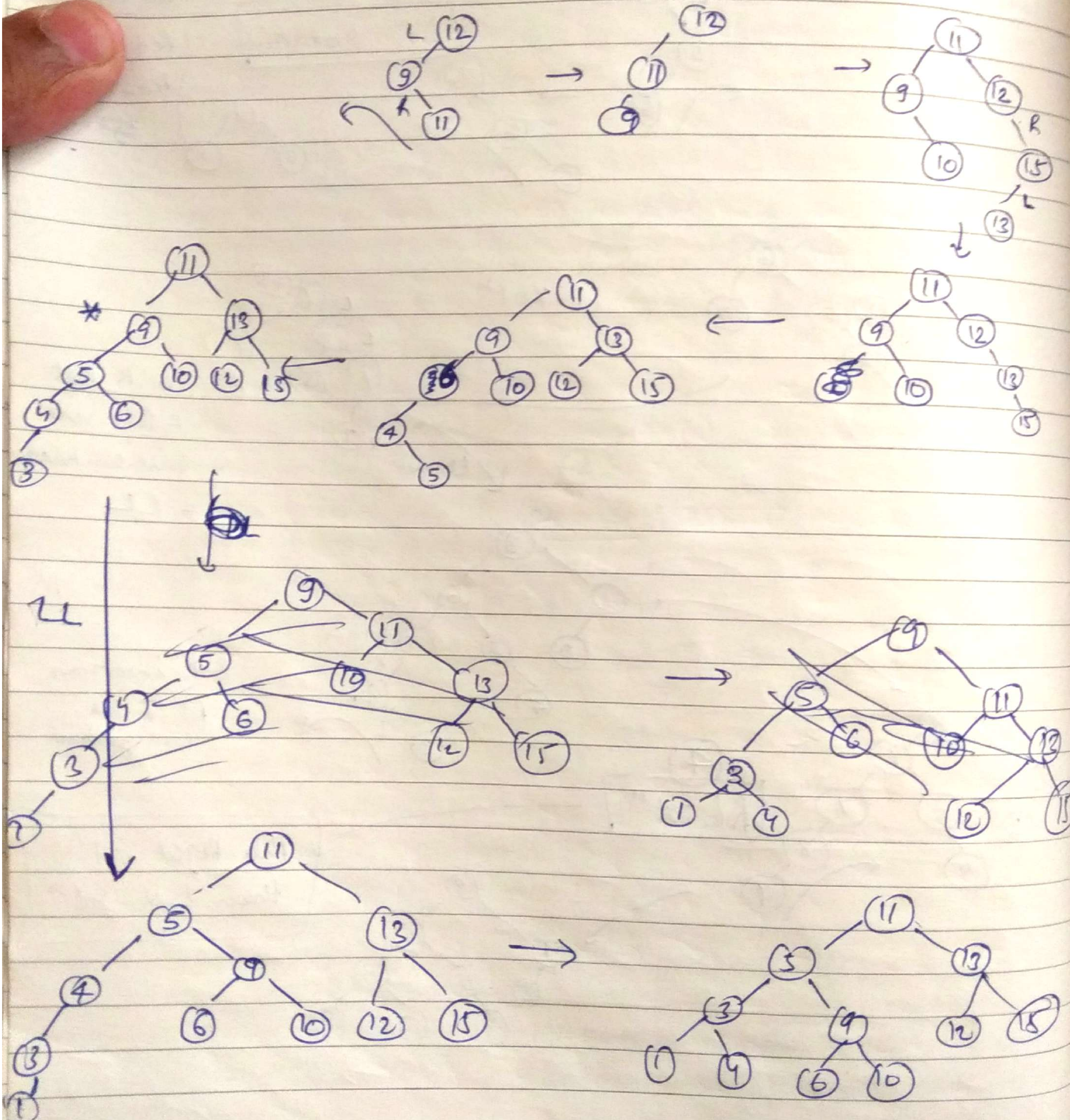
Complexity of finding smallest element in Balanced BST = log(n) (left most element)

" " largest " " = log(n) (Right most element)

" " smallest " BST = n



Q Draw AVL for 12 9 11 10 15 13 6 4 5 3 1



सूर्योदय 6:14 Sun Rise 6 : 14 सूर्यास्त 5:46 Sun Set 5 : 46

Prog to find height of tree

Height of node
= 1 + (max(LC, RC))
LC = left child's height
RC = Right child height

```
struct tree
{ int data;
  struct tree * left;
  struct tree * right;
}

int height ( struct tree * n )
  {
    if ( n->left == NULL & n->right == NULL ) //leaf
          { return 0; }
  else if ( n-> left == NULL )
        {return 1 + height ( n-> right ); }
  else if ( n-> right == NULL )
        {return 1 + height ( n->left ); }
  else
    {return 1 + max ( height (n->left), height (n->right)); }
  }
```

Prog to count no of leaf Nodes

```
int leaf ( struct tree * n)
    {
      if ( n->left == NULL & r->right == Null)
```

2015

'निर्बल के बल राम'।
'God is the help of the helpless.'

05 MARCH THURSDAY मार्च गुरुवार

Week 10 (064-301) सप्ताह १० (०६४-३०१)

विक्रम सम्वत् २०७१ फाल्गुन शुक्ल १४ शक सम्वत् १९३६ फाल्गुन १४ | Vikram Samvat 2071 Phalgun Shukla 14 Shak Samvat 1936 Phalgun 14

सूर्योदय ६:१४ Sun Rise 6 : 14 होलिकादहन : पूर्णिमा सूर्यास्त ६:४६ Sun Set 6 : 46

```
return 1;
else if ( n -> left == Null)
    return leaf (n -> right);
else if (n -> right == Null)
    return leaf ( n -> left );
else
    return leaf ( n -> left ) + leaf (n -> right);
```

No of Nodes

```
return 1;
return 1+ leaf ( n -> right) ;
return 1 + leaf ( n -> left ) ;
return leaf (n -> left) + leaf (n -> right);
```

No of internal nodes

```
return 0;
| rest all same
```

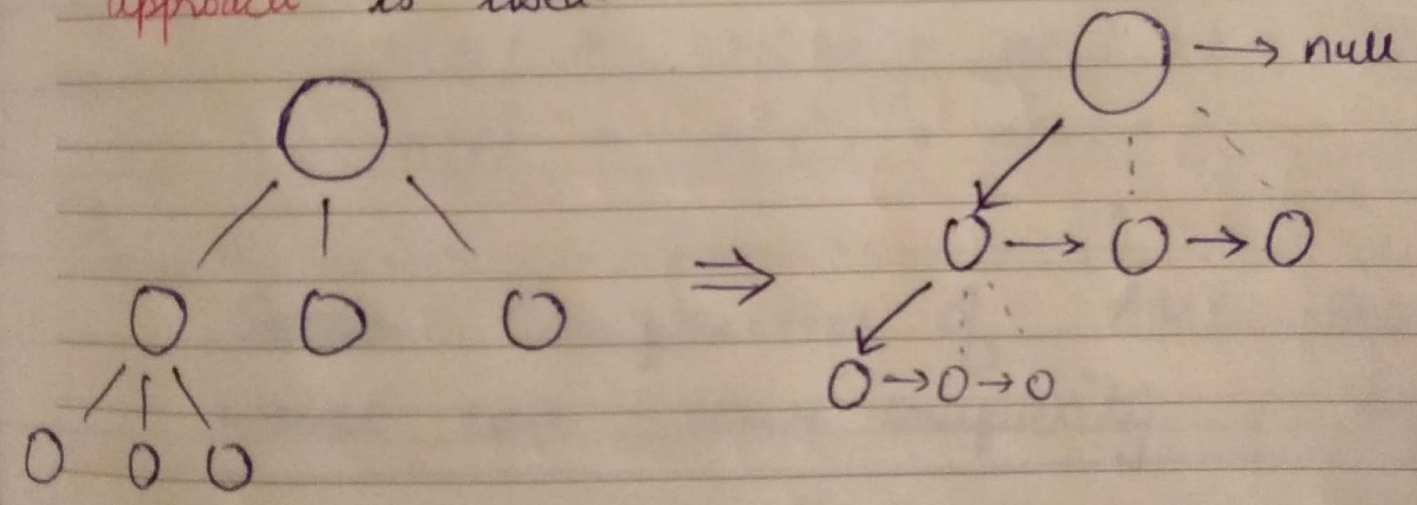
Inorder

```
void inorder ( struct tree * n)
{ if (n != Null)
    { inorder (n -> left);
      print ( n -> data );
      inorder ( n -> right);
    }
}
```

2015

कैसी दुःख की बात है कि मनुष्य जानता है तो भी गिरना पसंद करता है।
What a pity that even though a man understands, he still prefers to fall.

* Complexity of tree traversal = Order(n) ⇒ all the nodes are to be traversed
* complexity of finding smallest element in BST = O(h) ← height
* Worst case space complexity = O(h) = O(n) ← worst case = Chain
* for n-ary tree struct representation is bad coz we need n pointers for each child, better approach is first child and next sibling approach is used.



* Other way to store a tree is using ARRAY
  if P = parent
  L = 2p+1
  R = 2p+2

Smallest ——— BST = n   PAGE NO. DATE: Rajshree log(n)

07

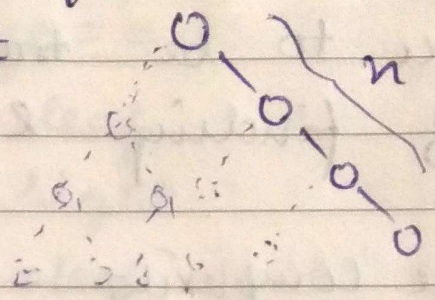
MARCH मार्च
SATURDAY शनिवार
Week 10 (066-299) सप्ताह १० (०६६-२९९)

विक्रम सम्वत् २०७१ Vikram Samvat
चैत्र कृष्ण २ Chaitra Krishna
शक सम्वत् १९३६ Shak Samvat
फाल्गुन १६ Phalgun 16

सूर्योदय ६:१२ Sun Rise 6 : 12   सूर्यास्त ५:४८ Sun Set

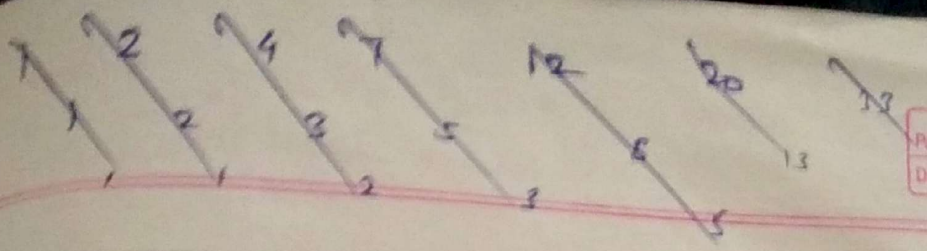
Q = Max array size required to store a tree with n

⇒ worst case =



⇒ $2^n - 1$ be array size

CS 46
07



Rajshree
PAGE NO
DATE

Max no of nodes in AVL tree of depth d = $2^{d+1} - 1$
Min no of —"— —"— —"— =

min no of nodes for depth (0) = 1 ⟹ O
—"— —"— (1) = 2 ⇒ O
O

min no of nodes for depth (d) = $\boxed{n(d) = n(d-1) + n(d-2) + 1}$

eg $n(5) = n(4) + n(3) + 1$ $12 + 7 + 1 = 20$
$n(4) = n(3) + n(2) + 1$ $7 + 4 + 1 = 12$
$n(2) = n(1) + n(0) + 1 = 2 + 1 + 1 = 4$
$n(3) = n(2) + n(1) + 1 = 4 + 2 + 1 = 7$
$n(6) = n(5) + n(4) + 1 = 20 + 12 + 1 = 33$

Complexities of AVL tree
Worst case search complexity = log(n)

complexity of insertion = complexity of searching where to put the element + complexity of rotⁿ

Root tk compare krna pdega ⇒ log(n) order (1) ⇒ const time required

⇒ Complexity of Insertion = log(n)

Deletion in AVL trees
→ same rules, after deletion, inorder successor or predecessor takes its place. and then perform rotations wherever required
delete krne k baad balance check krna pdega
⇒ Complexity of deletion in AVL = log(n)

In any complete binary tree if there are n internal nodes then there are n+1 external nodes

If depth starts from 1 then No of Nodes = $2^d - 1$

This is equivalent to Max no of nodes possible in binary tree

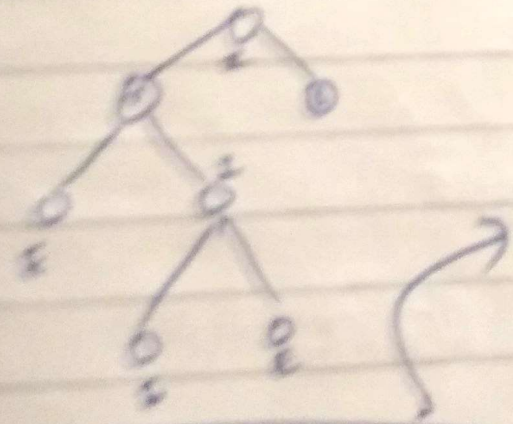
Max no of nodes in a binary tree = 0 — 0 — 0 = d

K-ary tree

Max nodes = $(k^{d+1} - 1)/(k-1)$

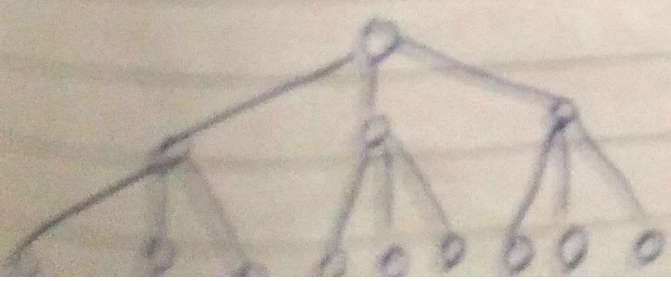
Min nodes = d

- Leaf nodes are External nodes, Non leaf are internal



In complete binary tree if there are n internal nodes then there are n+1 external nodes

4 External 3 Internal



4 External ; 4 internals